Reusing Locomotion Training Recipes Across Humanoids via Body-Aligned Virtual Joints

Sol Choi, Mingon Kim, Seokin Seo, Jongeun Choi, Donghyeon Kim, and Yisoo Lee

***Abstract*— Reinforcement learning has enabled robust humanoid locomotion, yet training recipes—rewards, architectures, hyperparameters—remain hard to reuse across robots with different native joint coordinates, since policies output native joint angles while action scales, bounds, gains, and penalties are defined in those robot-specific coordinates. Existing transmission models convert native targets into actuator commands but do not standardize the coordinate interface exposed to the policy. We introduce a *body-aligned virtual joint space* upstream of this mapping: policy actions and observations follow a common anatomical convention, and a deterministic kinematic layer maps virtual targets to native ones, leaving the existing transmission and low-level control stack unchanged. Under a single unmodified recipe on three humanoids with different leg-axis layouts, it is the only interface that tracks accurately, keeps the feet aligned with the travel direction, and trains stably on all three, and on Unitree G1 hardware it reduces heading drift by nearly a factor of six.**

I. Introduction

Reinforcement learning (RL) has enabled legged robots to acquire dynamic behaviors that transfer from simulation to hardware. Learned controllers have demonstrated agile quadrupedal locomotion [1], [2], traversal over challenging terrain [3], [4], rapid adaptation to changing dynamics [5], and training within minutes using massively parallel simulation [6]. Similar techniques have produced dynamic bipedal walking [7], stair traversal [8], and real-world humanoid locomotion [9], [10]. Nevertheless, adapting a controller to a robot with a different joint layout often requires substantial retraining and retuning by different training recipes.

As seen in Fig. 1, such kinematic diversity is often a deliberate consequence of mechanical design rather than an arbitrary modeling choice. Humanoid legs must generate large torques within tightly constrained volumes while keeping distal mass and inertia low. Designers therefore relocate actuators away from joint centers, employ linkage or parallel transmissions to obtain favorable leverage and distribute loads across actuators, and incline joint axes to improve workspace or locomotion-specific force transmission [11], [12], [13]. These choices can improve torque density and dynamic performance, but they also produce oblique, coupled, or configuration-dependent joint coordinates instead of a uniform body-aligned orthogonal arrangement.

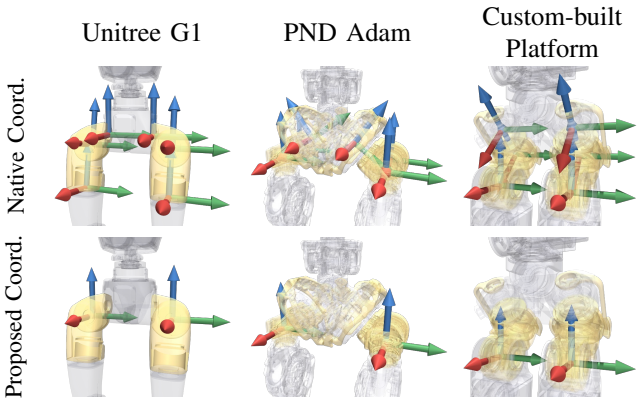


Fig. 1: Hip-joint coordinates of three representative humanoids. The native axes differ across platforms (top), whereas the proposed body-aligned virtual joint coordinates express the same joints about a shared body convention, giving a consistent axis layout across embodiments (bottom).

In this study, we focus on the hypothesis that part of this embodiment dependence originates from the coordinates in which policy actions are expressed. Most locomotion policies output desired angles in the robot's native joint space [2], [6], [7], [9]. Consider a body-aligned hip-roll motion. If the hip axes are aligned with the pelvis frame, the motion may correspond mainly to one native joint. With oblique axes, the same motion requires coordinated changes of several native joints, even when they are independently actuated. Such axes may be introduced for packaging, range-of-motion, and mechanical-design considerations [13]. The same task-level motion can therefore acquire a different numerical representation on each robot.

This issue is distinct from the conventional joint-to-actuator mapping. Four-bar linkages, parallel ankles, and other closed-chain transmissions are commonly handled by converting generalized joint targets into actuator commands [11], [12], [14]. These models describe how a robot-native joint motion is physically realized, but do not make the native coordinates themselves consistent across robots. Our method adds a virtual-to-native coordinate layer upstream of, rather than replacing, the existing transmission layer.

A sufficiently expressive policy can learn this relation implicitly, but then known embodiment-specific kinematics becomes embedded in the controller. A fixed orthogonal change of basis does not reduce the coverage of isotropic

*This work was supported by the...

S. Choi, S. Seo, and Y. Lee are with the Center for Humanoid Research, Korea Institute of Science and Technology, 02792, Seoul, South Korea

M. Kim is with the Advanced Robotics Lab., CTO Division, LG Electronics Inc., 06772 Seoul, South Korea

S. Choi and J. Choi are with the Department of Mechanical Engineering, Yonsei University, 03722, Seoul, South Korea

D. Kim is with the...

S. Choi and M. Kim contributed equally as co-first authors.

Yisoo Lee and Donghyeon Kim contributed equally as co-corresponding authors yisoo.lee@kist.re.kr.

exploration. Practical locomotion pipelines, however, are not fully coordinate invariant: they commonly use factorized action distributions and per-joint scaling, clipping, gains, limits, and regularization [7], [10], [15]. Changing the native coordinates thus changes the correlations, relative scales, saturation margins, and penalties associated with an equivalent body-level motion.

Prior work has shown that action representation affects locomotion learning. Reference-relative joint targets provide useful structure [16], task-space actions can improve learning efficiency [17], and torque actions exhibit different tuning and transfer characteristics from position targets [18]. Cross-embodiment methods instead seek generalization through graph-structured or modular policies [19], [20], morphology-conditioned architectures [21], [22], [23], and multi-embodiment training [24]. These approaches are complementary to ours: we standardize the physical meaning of the joint-level action before robot-specific coordinate and transmission mappings are applied.

The contributions are threefold: (i) we identify native-coordinate inconsistency—distinct from the joint-to-actuator transmission, which existing models already handle—as a source of embodiment dependence, and address it with an analytic coordinate layer derived from the kinematic model that maps actions, joint-velocity observations, and joint-space gains between a shared body-aligned space and each robot's native space; (ii) we show that where the map is inserted during training matters: applying it post-hoc avoids an instability that arises when it is composed in the training loop; and (iii) we evaluate the interface on three humanoids and on Unitree G1 hardware, where the benefit appears first in gait geometry rather than in aggregate task error.

II. Background and Problem Formulation

A. Native-Joint Policy Interface

Most reinforcement-learning locomotion controllers formulate the policy action in the generalized joint coordinates provided by the robot model [2], [6], [7], [9]. Let $\mathbf{q} \in \mathcal{Q}^N$ and $\dot{\mathbf{q}}$ denote the native joint configuration and velocity of a robot. Given an observation $\mathbf{o} \in \mathcal{O}$, the policy produces a desired joint configuration,

$$\mathbf{q}^{\mathrm{des}} \sim \pi_\theta\left(\cdot \mid \mathbf{o}\right), \qquad \mathbf{q}^{\mathrm{des}} \in \mathcal{Q}^N. \tag{1}$$

The policy output is expressed as a desired joint angle; any normalization, output scaling, or range restriction is included in the policy parameterization. The superscript N is used only for the configuration-space symbol Q to distinguish the native spaces, and is omitted from other symbols for notational simplicity in this paper.

For directly actuated joints, the target is commonly tracked by a joint-space proportional–derivative (PD) controller,

$$\boldsymbol{\tau} = \mathbf{K}_p\left(\mathbf{q}^{\mathrm{des}} - \mathbf{q}\right) - \mathbf{K}_d\dot{\mathbf{q}}, \tag{2}$$

where $\mathbf{K}_p$ and $\mathbf{K}_d$ are control gains. For robots with nontrivial transmissions, the desired generalized-joint configuration is first converted into an actuator control input $\mathbf{u}$,

$$\mathbf{u} = \Psi\left(\mathbf{x}, \mathbf{q}^{\mathrm{des}}\right), \tag{3}$$

where $\mathbf{x}$ contains the states required by the transmission and low-level controller. The map Ψ may be direct for a serial joint or nonlinear and configuration dependent for a four-bar linkage, parallel ankle, or another closed-chain mechanism [11], [12], [14]. In either case, the policy action remains defined in the generalized joint coordinates chosen for that robot.

This coordinate choice also shapes the learning problem: diagonal Gaussian action distributions [7], action scales and bounds, servo gains, joint limits, and regularization are all defined per native joint, and none of these components is invariant to a change of joint coordinates.

B. Problem Formulation

Consider robot platforms r and s with native joint spaces $\mathcal{Q}_r^N$ and $\mathcal{Q}_s^N$. Even when the robots have the same joint dimensions, their native coordinates may represent different rotation axes and axis orderings, as illustrated in Fig. 1. Let $\Delta\mathcal{M}_r(\mathbf{q}_r, \Delta\mathbf{q})$ denote the body-relative link motion induced by a native-coordinate perturbation $\Delta\mathbf{q}$. In general,

$$\Delta\mathcal{M}_r(\mathbf{q}_r, \Delta\mathbf{q}) \neq \Delta\mathcal{M}_s(\mathbf{q}_s, \Delta\mathbf{q}). \tag{4}$$

Thus, the same policy output can have different physical effects across robots. A policy acting directly in native coordinates must therefore learn not only the locomotion task but also the robot-specific coordinate combinations required to produce task-relevant link motions. Moreover, per-joint action distributions, output bounds, control gains, and regularization make the resulting learning problem dependent on the native coordinate convention.

The effects of joint geometry and native coordinates are not always visible in aggregate task metrics. With the same reward formulation, policies may achieve similar velocity tracking while producing asymmetric steps, irregular foot trajectories, or unnatural leg motions. Because such artifacts are poorly captured by standard scalar metrics, engineers often add and tune robot-specific reward terms through repeated inspection of simulation or hardware rollouts, limiting the practical reuse of a common training recipe.

Our objective is not identical whole-body motions, which also depend on geometry and dynamics, but a common interface in which corresponding policy dimensions represent the same body-aligned rotations with locally consistent physical effects, while preserving the action dimension, joint-target structure, and each robot's existing control stack.

III. Proposed Method

A. Architecture of Body-Aligned Virtual Joint Interface

To address the coordinate inconsistency formulated in Sec. II-B, we introduce a common body-aligned virtual joint space and a robot-specific coordinate interface. For each robot platform r, the policy outputs a desired virtual joint configuration, which is mapped to native joint targets before being passed to the robot's existing actuation interface. Importantly, the virtual coordinates are not modeled as additional DoFs in simulation. Because the virtual–native relation is determined by the robot kinematics and branch

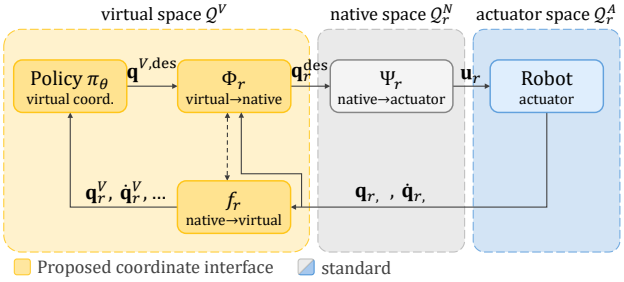


Fig. 2: Overview of the proposed body-aligned virtual joint interface. The proposed modules map policy actions and joint-state observations between the shared virtual space and each robot's native space, while the existing native-to-actuator control stack remains unchanged.

convention, it is realized as a deterministic interface rather than as extra simulated degrees of freedom, which preserves the original dynamics and constraints and allows the same mapping to be used on hardware.

1) Canonical humanoid-leg convention: Let $\mathcal{F}_B$ denote the robot body frame, whose x-, y-, and z-axes point forward, leftward, and upward, respectively. For the common humanoid-leg topology, we define a canonical 3–1–2 coordinate structure consisting of a 3-DoF hip, a 1-DoF knee, and a 2-DoF ankle:

$$\mathbf{q}^V_{\mathrm{leg}} = \begin{bmatrix} \psi_{\mathrm{hip}} & \phi_{\mathrm{hip}} & \theta_{\mathrm{hip}} & \theta_{\mathrm{knee}} & \phi_{\mathrm{ankle}} & \theta_{\mathrm{ankle}} \end{bmatrix}^\top. \quad (5)$$

Here, (ψ, ϕ, θ) denote yaw, roll, and pitch about the body-aligned z-, x-, and y-axes. The corresponding orientation conventions are

$$\begin{aligned} \mathbf{R}^V_{\mathrm{hip}} &= \mathbf{R}_z(\psi_{\mathrm{hip}})\mathbf{R}_x(\phi_{\mathrm{hip}})\mathbf{R}_y(\theta_{\mathrm{hip}}), \\ \mathbf{R}^V_{\mathrm{ankle}} &= \mathbf{R}_x(\phi_{\mathrm{ankle}})\mathbf{R}_y(\theta_{\mathrm{ankle}}). \end{aligned} \quad (6)$$

Ankle yaw is not independently parameterized. For the knee, the nominal standing configuration is defined as zero, and the coordinate sign is chosen such that positive rotation corresponds to knee flexion.

2) Link-based virtual-frame construction: For each multi-DoF group $g \in \{\mathrm{hip}, \mathrm{ankle}\}$, let P_g and D_g denote the proximal and distal links bounding its native joint chain. These are the pelvis and thigh for the hip, and the shank and foot for the ankle. A virtual reference frame is rigidly attached to each link. At the nominal configuration $\mathbf{q}^0_r$, both frames are aligned with $\mathcal{F}_B$; thereafter, they move with their respective links. Their relative orientation therefore describes thigh motion relative to the pelvis or foot motion relative to the shank in a common body-aligned convention.

For a URDF-based implementation, let ${}^B\mathbf{R}_{r,X}(\mathbf{q}_r)$ denote the orientation of link frame X obtained by forward kinematics. The constant calibration offsets are

$$\mathbf{C}_{r,X} = \left({}^B\mathbf{R}_{r,X}(\mathbf{q}^0_r)\right)^\top, \qquad X \in \{P_g, D_g\}, \quad (7)$$

and the calibrated relative orientation is

$$\begin{aligned} \overline{\mathbf{R}}_{r,g}(\mathbf{q}_{r,g}) = &\left({}^B\mathbf{R}_{r,P_g}(\mathbf{q}_r)\mathbf{C}_{r,P_g}\right)^\top \\ &{}^B\mathbf{R}_{r,D_g}(\mathbf{q}_r)\mathbf{C}_{r,D_g}. \end{aligned} \quad (8)$$

By construction, $\overline{\mathbf{R}}_{r,g}(\mathbf{q}^0_{r,g}) = \mathbf{I}$. Although the forward kinematics may be evaluated using the complete configuration $\mathbf{q}_r$, the relative orientation depends only on the joints between P_g and D_g. Because only orientation is used, the link-frame origins do not affect the virtual coordinates. The same construction applies to any kinematic model that provides the required link transforms.

3) Native–virtual coordinate encoder: For compactness, let $\mathcal{G} = \{\mathrm{hip}, \mathrm{knee}, \mathrm{ankle}\}$, with virtual dimensions 3, 1, and 2, respectively. Let $\mathcal{Q}^N_{r,g}$ and $\mathcal{Q}^V_g$ denote the native and virtual configuration spaces of group g. The robot kinematics define an encoding map

$$f_{r,g} : \mathcal{Q}^N_{r,g} \to \mathcal{Q}^V_g, \qquad f_{r,g}(\mathbf{q}^0_{r,g}) = \mathbf{0}. \quad (9)$$

For the hip and ankle, $f_{r,g}$ extracts the coordinates of $\overline{\mathbf{R}}_{r,g}$ according to Eq. (6); for the knee, it applies only the calibrated zero offset and sign. Thus, corresponding virtual coordinates have the same anatomical interpretation across robots, without requiring identical native joint layouts.

4) Insertion point of the interface: The interface can be introduced at two different points of the pipeline, and we distinguish them explicitly because they lead to different optimization behavior. In the *in-loop* form, f_r and Φ_r are evaluated at every control step of training on the unmodified native model, so the policy interacts with the native kinematics through the coordinate layer from the first iteration. In the *post-hoc* form, which is the configuration proposed in this work, training is instead carried out on a body-aligned model of the same robot—the kinematic model with each leg joint axis replaced by the corresponding canonical axis of Eq. (5), leaving link masses, inertias, and joint limits unchanged—so that native and virtual coordinates coincide during training and no coordinate map is evaluated in the loop. The maps f_r and Φ_r are then applied at deployment to run the resulting policy on the native robot. Both forms share the virtual action semantics and deployment stack; the post-hoc form instead trains on an idealized model and absorbs the resulting mismatch as a deployment-time disturbance, which Sec. IV-B-3 shows is benign relative to the instability of the in-loop form.

B. Orientation-Based Native Mapping

This subsection considers the nontrivial hip and ankle mappings; the calibrated knee map is one-dimensional and affine. Given a desired virtual configuration $\mathbf{q}^{V,\mathrm{des}}_g$, the corresponding native target is defined by orientation matching:

$$\overline{\mathbf{R}}_{r,g}\left(\mathbf{q}^{\mathrm{des}}_{r,g}\right) = \mathbf{R}^V_g\left(\mathbf{q}^{V,\mathrm{des}}_g\right). \quad (10)$$

When the native joint axes span the group's rotational DoF, so that the matched orientation maps to a unique joint-coordinate vector, the native target is obtained directly as

$$\mathbf{q}^{\mathrm{des}}_{r,g} = \mathrm{coord}_{r,g}\left[\mathbf{R}^V_g\left(\mathbf{q}^{V,\mathrm{des}}_g\right)\right] \equiv \Phi_{r,g}\left(\mathbf{q}^{V,\mathrm{des}}_g\right), \quad (11)$$

where $\mathrm{coord}_{r,g}(\cdot)$ extracts the native joint coordinates according to the robot-specific axis arrangement and joint ordering. When multiple solutions exist, the branch continuous with the nominal configuration is selected. The

relation between finite native and virtual angles is generally nonlinear; hence, no constant linear transformation between their joint-angle vectors is assumed.

C. Differential Mapping by Full-Coordinate Jacobian

The group-wise encoders are assembled into a single robot-level coordinate interface:

$$\mathbf{q}^V = f_r(\mathbf{q}_r) = \mathrm{col}_{g\in\mathcal{G}}\left[f_{r,g}(\mathbf{q}_{r,g})\right]. \tag{12}$$

Groups that already follow the virtual convention are represented by identity components of f_r. The corresponding coordinate Jacobian is

$$\mathbf{J}_r(\mathbf{q}_r) = \frac{\partial f_r}{\partial \mathbf{q}_r}, \qquad \dot{\mathbf{q}}^V = \mathbf{J}_r(\mathbf{q}_r)\dot{\mathbf{q}}_r. \tag{13}$$

Because each virtual group depends only on its corresponding native group,

$$\mathbf{J}_r(\mathbf{q}_r) = \mathrm{blkdiag}_{g\in\mathcal{G}}\left[\mathbf{J}_{r,g}(\mathbf{q}_{r,g})\right]. \tag{14}$$

For a single leg:

$$\mathbf{J}_r = \mathrm{blkdiag}\left(\mathbf{J}_{r,\mathrm{hip}}, s_{r,\mathrm{knee}}, \mathbf{J}_{r,\mathrm{ankle}}\right), \tag{15}$$

where $\mathbf{J}_{r,\mathrm{hip}} \in \mathbb{R}^{3\times3}$, $\mathbf{J}_{r,\mathrm{ankle}} \in \mathbb{R}^{2\times2}$, and $s_{r,\mathrm{knee}} \in \{-1, 1\}$ accounts for the calibrated direction.

For each multi-DoF group, let $\mathbf{E}_g^V(\mathbf{q}_g^V)$ and $\mathbf{E}_{r,g}(\mathbf{q}_{r,g})$ map virtual and native joint rates to the same angular velocity expressed in the proximal-link frame:

$$\boldsymbol{\omega}_g^P = \mathbf{E}_g^V(\mathbf{q}_g^V)\dot{\mathbf{q}}_g^V = \mathbf{E}_{r,g}(\mathbf{q}_{r,g})\dot{\mathbf{q}}_{r,g}. \tag{16}$$

When $\mathbf{E}_g^V$ has full column rank,

$$\mathbf{J}_{r,g}(\mathbf{q}_{r,g}) = \left[\mathbf{E}_g^V\left(f_{r,g}(\mathbf{q}_{r,g})\right)\right]^\dagger \mathbf{E}_{r,g}(\mathbf{q}_{r,g}), \tag{17}$$

where $(\cdot)^\dagger$ denotes the Moore–Penrose pseudoinverse. For a 3-DoF rotational group away from a representation singularity, it reduces to the ordinary inverse. For the 2-DoF ankle, the relation is defined on its admissible rotational subspace.

For the yaw–roll–pitch hip convention,

$$\mathbf{E}_{\mathrm{hip}}^V = \left[\mathbf{e}_z,\ \mathbf{R}_z(\psi)\mathbf{e}_x,\ \mathbf{R}_z(\psi)\mathbf{R}_x(\phi)\mathbf{e}_y\right], \tag{18}$$

whose determinant is $\det \mathbf{E}_{\mathrm{hip}}^V = \cos\phi_{\mathrm{hip}}$. The representation becomes singular only at $\phi = \pm90^\circ$, where the yaw and pitch axes are collinear. Choosing roll as the middle rotation places this singularity outside the hip-abduction range encountered during normal locomotion. Robot-specific mechanism singularities may additionally enter through $\mathbf{E}_{r,g}$.

The coordinate Jacobian can also be used to map a common virtual-space stiffness to the native coordinates. Around the nominal configuration,

$$\mathbf{K}_r^{\mathrm{full}} = \mathbf{J}_r^\top(\mathbf{q}_r^0)\mathbf{K}^V\mathbf{J}_r(\mathbf{q}_r^0). \tag{19}$$

For independent joint gains, we use its diagonal approximation $\mathbf{K}_r = \mathrm{Diag}(\mathrm{diag}(\mathbf{K}_r^{\mathrm{full}}))$. The same mapping applies to damping gains.

The full Jacobian is also used to transform joint-velocity observations. When an explicit decoder is unavailable for a particular mechanism, it may support a damped local numerical inverse. The standard pseudoinverse is used in regular configurations, with damping introduced only near rank-deficient configurations.

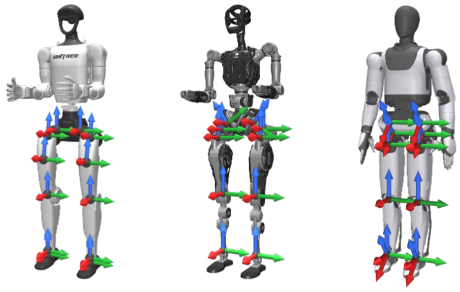


Fig. 3: Humanoid platforms used in simulation—Unitree G1, PND Adam, and a custom-built platform—with their leg joint-axis configurations.

D. Policy, Observation, and Actuation Integration

The group-wise decoders are assembled into the full map Φ_r, with identity mappings for unaffected joints. The complete action path is

$$\begin{aligned}\mathbf{q}_r^{\mathrm{des}} &= \Phi_r\left(\mathbf{q}^{V,\mathrm{des}}; \mathbf{q}_r\right),\\ \mathbf{u}_r &= \Psi_r\left(\mathbf{x}_r, \mathbf{q}_r^{\mathrm{des}}\right),\end{aligned} \tag{20}$$

where Ψ_r is the robot's existing native-joint-to-actuator interface and $\mathbf{x}_r$ contains any state required by the existing controller. The current configuration is included in Φ_r only when needed for branch selection or a configuration-dependent inverse. The transmission model, low-level controller, safety checks, and physical joint limits remain unchanged. Closed-chain mechanisms such as four-bar linkages and parallel ankles, present on the evaluated platforms, are handled entirely within Ψ_r as in the native pipeline; the proposed layer acts upstream on the joint-coordinate semantics and is independent of how those native targets are physically realized.

Measured joint states are encoded using the same convention:

$$\mathbf{q}_{r,g}^V = f_{r,g}(\mathbf{q}_{r,g}), \qquad \dot{\mathbf{q}}_{r,g}^V = \mathbf{J}_{r,g}(\mathbf{q}_{r,g})\dot{\mathbf{q}}_{r,g}. \tag{21}$$

Joint-position and joint-velocity observations, action bounds, and action-related regularization are expressed in virtual coordinates. Native joint limits and actuator saturation are enforced after the mapping in their physical spaces. The proposed and native-coordinate controllers therefore retain the same policy architecture and action dimension; only the joint-coordinate interface is changed.

IV. Simulation and Experimental Evaluation

We evaluate whether the proposed body-aligned interface enables a single locomotion recipe to generalize across humanoids with different native joint layouts. We compare it with native- and task-space baselines in simulation and assess transfer to hardware.

A. Platforms and Experimental Setup

1) Platforms and Training Setup: We evaluate the proposed interface on three humanoid platforms with distinct leg kinematics: the Unitree G1, the PND Adam, and a custom-built platform (Fig. 3). Each platform is used in its 12-DoF bipedal lower-body configuration, with six joints per leg following the canonical 3–1–2 topology. For every platform, the body-aligned virtual coordinates follow the canonical body convention. The per-robot encoder f_r, decoder Φ_r (Sec. III-A), and Jacobian matrix are derived analytically from the URDF model.

All configurations share a single training recipe, implemented as a manager-based velocity-tracking environment with GPU-parallel simulation in Isaac Lab [6]. Unless otherwise specified, the network architecture, PPO hyperparameters, observations, commands, rewards, curriculum, events, and simulation settings follow the official Unitree RL Lab G1-29DoF velocity-tracking configuration [25]. To construct the 12-DoF leg-only controller used in this study, the arm and waist joints and their associated reward terms are removed, and the upper-body joints are fixed at their nominal initial configuration during training. A gait-phase clock is added only to the critic observation, while all remaining settings are kept unchanged. During sim-to-sim and hardware deployment, the upper-body joints are controlled independently to maintain the same initial posture. Across all three robot platforms and coordinate configurations, only the joint-coordinate interface used to express the policy actions and joint-state observations is varied, thereby isolating the effect of the proposed coordinate representation.

2) Baselines and Evaluation Protocol: Against this fixed recipe we compare four joint-coordinate interfaces. (i) *Native joint space* is the conventional baseline, in which the policy acts and observes directly in each robot's native joint coordinates. (ii) *Task space* follows Duan et al. [17]: the policy outputs foot-position residuals and direct hip-yaw and foot-pitch targets, which are converted to joint-level commands by a model-based task-space controller. The remaining two interfaces both use the body-aligned virtual joint space and differ only in where the coordinate map is inserted (Sec. III-A): (iii) *Virtual joint space (in-loop)*, in which the analytic coordinate map is applied on the unmodified native model throughout training, and (iv) *Virtual joint space (post-hoc)*, in which the policy is trained on a body-aligned model and the coordinate map is applied to the native robot afterward. Note that *Virtual joint space (post-hoc)* is the proposed method in this study.

All reported metrics are measured under sim-to-sim transfer: each trained policy is evaluated closed-loop in MuJoCo [26], whose physics and contact model differ from the Isaac Lab training environment. The full deployment stack runs unchanged: Φ_r decodes virtual targets to native targets, which each robot's PD controller and transmission convert to actuator commands. Results are averaged over diverse velocity commands and $n = 3$ seeds.

TABLE I: Sim-to-sim velocity tracking on the constant velocity command ($n = 3$ seeds, mean$\pm$std). Velocity error and lateral drift are expressed in the base heading frame.

Robot	Interface	Velocity Error (m/s)	Lateral Drift (m)
Unitree G1	Native	$\mathbf{0.042 \pm 0.001}$	0.086 ± 0.001
	Task space	0.048 ± 0.002	0.129 ± 0.003
	Virtual (in-loop)	$\mathbf{0.042 \pm 0.007}$	$\mathbf{0.084 \pm 0.001}$
	Virtual (post-hoc)	0.053 ± 0.018	0.089 ± 0.003
PND Adam	Native	0.106 ± 0.018	0.094 ± 0.007
	Task space	0.207 ± 0.016	0.082 ± 0.010
	Virtual (in-loop)	0.143 ± 0.034	0.113 ± 0.013
	Virtual (post-hoc)	$\mathbf{0.085 \pm 0.018}$	$\mathbf{0.075 \pm 0.006}$
Custom-built Platform	Native	0.032 ± 0.005	0.042 ± 0.002
	Task space	0.477 ± 0.333	0.120 ± 0.070
	Virtual (in-loop)	0.033 ± 0.005	0.040 ± 0.003
	Virtual (post-hoc)	$\mathbf{0.029 \pm 0.002}$	$\mathbf{0.033 \pm 0.005}$

B. Learning Performance Across Joint Layouts

1) Velocity Tracking and Lateral Drift: Under the single unmodified recipe, all four interfaces learn to track the commanded velocities on all three robots. We evaluate tracking under sim-to-sim transfer using a predefined set of 25 constant velocity-command combinations spanning $v_x \in [-0.5, 1.0]\,\mathrm{m/s}$, $v_y \in [-0.3, 0.3]\,\mathrm{m/s}$, and $\omega_z \in [-0.2, 0.2]\,\mathrm{rad/s}$. The results are averaged over all 25 commands, reporting the velocity-tracking error. Additional sim-to-sim tests evaluate lateral drift, defined as the magnitude of the lateral displacement accumulated 20 s rollout in the initial base-heading frame under a straight command ($v_x = 0.5$, $v_y = \omega_z = 0$).

As can be seen in Table I, on the two obliquely-hipped platforms, Adam and the custom robot, the proposed interface attains both the lowest tracking error and the smallest lateral drift of all interfaces; on G1, whose native hip carries only a small ($\sim 10^\circ$) mounting offset and is therefore already close to the body convention, the interfaces track comparably, with the differences falling within the seed-to-seed spread and leaving little for the coordinate change to correct. The task-space baseline instead degrades on Adam and fails on the custom platform, where it barely advances under the command ($0.48\,\mathrm{m/s}$ error). The takeaway is not that the proposed interface dominates everywhere, but that a single unchanged recipe attains accurate, low-drift tracking across all three embodiments, whereas each baseline degrades on at least one platform.

2) Gait-Pattern Consistency: Accurate velocity tracking does not necessarily imply a well-aligned gait motion. Policies can satisfy the commanded velocity while producing unintended and visually unnatural gait patterns, such as persistent toe-in or toe-out, particularly with oblique native hip axes. We therefore evaluate foot alignment using the foot-progression angle (FPA), defined as the foot heading relative

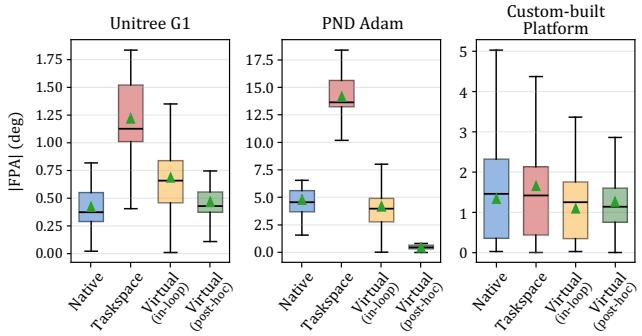


Fig. 4: Foot-progression angle (toe-out) by platform and coordinate interface (all seeds, both feet).

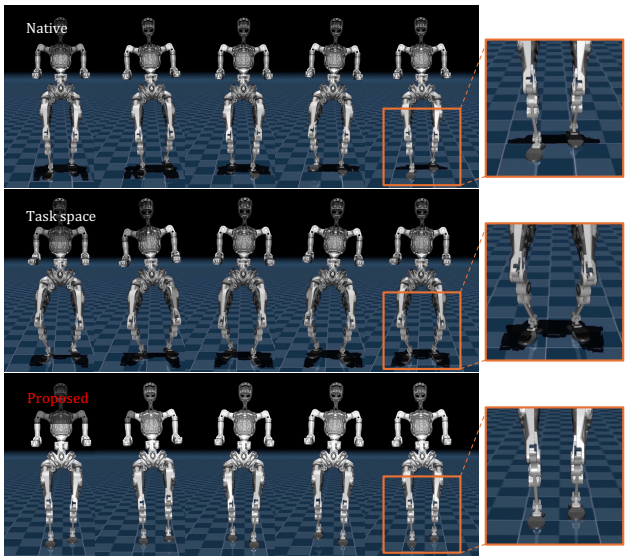


Fig. 5: Swing-leg motion on the PND Adam in simulation under the native, proposed, and task-space interfaces. The proposed interface keeps the feet aligned with the direction of travel.

to the base heading during mid-stance:

$$\begin{aligned} \mathrm{FPA} &= \mathrm{atan2}\big(r^f_{21},\, r^f_{11}\big) - \psi_b, \\ \overline{|\mathrm{FPA}|} &= \frac{1}{2} \sum_{f\in\{L,R\}} \big\langle |\mathrm{FPA}_f| \big\rangle, \end{aligned} \tag{22}$$

where $\mathbf{R}_f = [r^f_{ij}]$ is the foot rotation matrix, ψ_b is the base heading, and $\langle\cdot\rangle$ denotes averaging over mid-stance samples and gait cycles. A value close to 0° indicates that the feet remain aligned with the body's forward direction.

The proposed interface keeps the feet essentially aligned with the body on every platform ($\overline{|\mathrm{FPA}|} \le 1.3^\circ$), whereas the baselines exhibit a strongly *embodiment-dependent* toe-out (Fig. 4). The effect is largest on the Adam platform. Its hip roll and yaw axes are pelvis-aligned at the nominal pose, but the *proximal* hip-pitch axis is inclined by 35° about the fore–aft (x) axis, with mirrored signs on the two legs. Because the tilted axis is both the most proximal joint of the chain and the one carrying the largest excursion in walking, a native hip-pitch command of θ injects $\sin(35^\circ)\,\theta \approx 0.57\theta$ of body yaw of opposite sign on each leg—at a typical 23° swing, 13° of thigh yaw—producing an outward splay that native coordinates can cancel only by learning the axis mixing implicitly. The native and task-space interfaces produce mean foot-progression angles of 4.8° and 14.2°, respectively, compared with 0.5° for the proposed interface, as also seen qualitatively in Fig. 5. On G1 and the custom platform the mounting tilt is instead about the lateral (y) axis and shared by both legs; it leaves the pitch axis invariant and mixes only roll and yaw, whose excursions are small in steady walking, so toe-out is small for all interfaces ($\le 1.7^\circ$).

The proposed interface therefore does not reduce toe-out universally; rather, it prevents the pronounced outward foot misalignment induced by native coordinates on platforms with obliquely mounted hips.

Foot alignment alone does not capture temporal coordination or step-to-step repeatability. Let t^f_k denote the k-th lift-off of foot $\{L, R\}$, and let τ^R_k be the first right-foot lift-off following t^L_k. We measure gait-period variability using the coefficient of variation $\mathrm{CV}(T)$ and deviation from ideal left–right alternation as

$$e_\Phi = 100 \left\langle \left| \frac{\tau^R_k - t^L_k}{\widetilde{T}} - \frac{1}{2} \right| \right\rangle_k, \tag{23}$$

where $\widetilde{T}$ is the median gait period. Thus, $e_\Phi = 0$ indicates exact alternation. All metrics are averaged over steady-state rollouts from three seeds.

The proposed interface provides more consistent inter-leg coordination than native coordinates across all three platforms. It reduces the phase error from 1.62% to 0.23% on G1 and from 1.64% to 0.54% on the custom platform, while reducing it only marginally on Adam (from 2.59% to 2.36%). Thus, although the improvement on Adam is modest, the proposed interface consistently matches or outperforms native coordinates without platform-specific tuning. The other interfaces exhibit more pronounced embodiment-dependent behavior. On the custom platform, the task-space baseline reaches $\mathrm{CV}(T) = 11.4\%$, whereas the proposed and native-coordinate interfaces remain below 0.6%. On Adam, one seed of the in-loop variant converges to a substantially shorter gait period (0.668 s versus approximately 0.81 s for the other seeds), consistent with its unstable training behavior. The task-space baseline also converges to a near-extended-knee gait on G1 and the custom platform, indicating a further platform-dependent failure under the shared training recipe.

These gait-shape differences are not penalized by a velocity-tracking objective, yet in practice they are exactly what practitioners suppress through manual, often platform-specific, reward shaping. Because the proposed interface produces body-consistent gaits as an emergent property of the coordinate representation rather than of a tuned reward, it removes a recurring and time-consuming step from the locomotion-learning workflow—a benefit that, in our experience, is as valuable as the tracking results themselves.

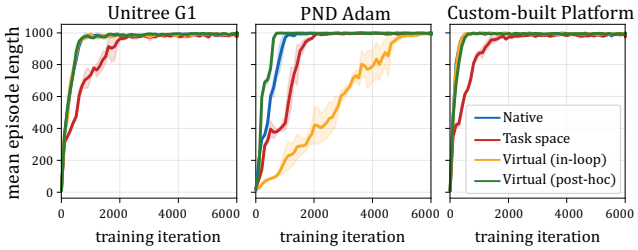


Fig. 6: Training stability across the three embodiments, measured by mean episode length over the first 6k iterations (mean ± standard deviation over three seeds). The dotted line marks half of the maximum episode length.

3) Training Robustness Across Embodiments: We evaluate training robustness using episode-length stability rather than wall-clock time, since the runs used different GPU resources. The instability fraction is defined as the proportion of training iterations, counted after the mean episode length first reaches half of its maximum, in which it subsequently falls back below that level; the initial learning transient is therefore excluded. We do not compare raw rewards because the coordinate interfaces alter the scales of action-dependent penalties.

As shown in Fig. 6, the proposed interface trains consistently across all three embodiments, remaining in the unstable regime for at most 0.1% of the training iterations. In contrast, the in-loop variant reaches an instability fraction of 15.3% on Adam, while the task-space baseline remains unstable for 1.2–6.0% of training across the three platforms. These results show that training on the body-aligned model and applying the coordinate map afterward avoids the embodiment-dependent optimization instability observed with the alternative interfaces.

C. Hardware Experiments

1) Deployment Feasibility and Path Regularity: We deployed the native, in-loop, and proposed interfaces on the Unitree G1 without reward retuning, gain adjustment, or coordinate-specific modification. Each controller was commanded to walk straight at $v_x = 0.5\,\mathrm{m/s}$ with $v_y = \omega_z = 0$, over 6 rollouts each.

All three interfaces produced stable walking, but differed substantially in their ability to maintain the commanded heading. The native and in-loop controllers drifted toward the edge of the test area and were stopped after $5.1 \pm 1.7\,\mathrm{m}$ and $6.9 \pm 3.0\,\mathrm{m}$, respectively. In contrast, every rollout of the proposed interface reached the end of the available walkway, traveling $13.8 \pm 1.1\,\mathrm{m}$; this distance was therefore limited by the test area rather than controller failure.

Because base position was not available from the deployment logs, heading drift was quantified using the absolute IMU yaw rate over an identical 5-s steady-walking interval. The proposed interface reduced the yaw rate to $0.89 \pm 0.54^\circ/\mathrm{s}$, compared with $5.31 \pm 1.90^\circ/\mathrm{s}$ for native coordinates and $4.08 \pm 1.02^\circ/\mathrm{s}$ for the in-loop interface. Although the interfaces exhibited similar aggregate tracking performance on G1 in simulation, the hardware results reveal a clear difference in their ability to preserve the commanded heading under the sim-to-real gap.

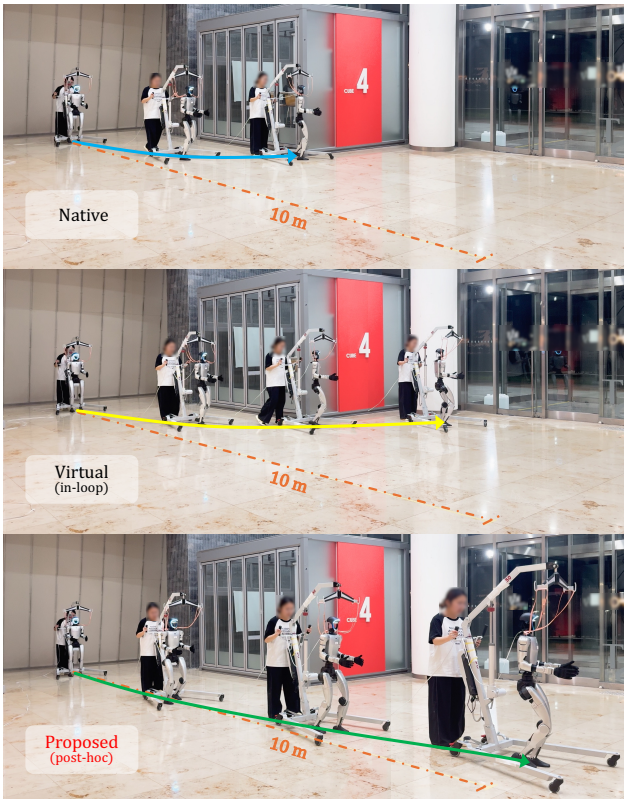


Fig. 7: Hardware rollouts under a constant straight-walking command, composited from video frames of one representative rollout per interface. The dash-dotted line is a $10\,\mathrm{m}$ floor reference along the commanded heading; the solid curve traces the path walked.

The separation on hardware, absent from the aggregate sim-to-sim tracking of Table I, admits a consistent reading. Heading drift is a signed, accumulating quantity: a small out-of-plane component of the swing foot injects a yaw impulse of the same sign at every step, so a bias that a magnitude-averaged velocity error cannot resolve integrates into meters of deviation over a $10\,\mathrm{m}$ walkway (Fig. 7). The separation is in fact already present in simulation, but confined to gait geometry—the inter-leg phase error on the same platform falls from 1.62% to 0.23%—rather than to task error. What changes on hardware is the perturbation acting on the coordinate layer itself. Joint-axis calibration residuals, mounting tolerances, transmission compliance, and load-dependent tracking lag each insert a small quasi-static rotation between the commanded and the realized link orientation. Expressed in obliquely mounted native axes, such a discrepancy gives a purely sagittal intent a systematic abduction and yaw component, which the policy can only offset through the axis mixing it has implicitly learned from the nominal simulated geometry—precisely the assumption the real robot

violates. The proposed interface instead supplies this mixing analytically through f_r and Φ_r, leaving the residual as an unmodeled disturbance on a body-aligned action rather than a bias in a learned compensation. A controlled attribution would require hardware ablations beyond these rollouts.

2) Onboard Computational Cost: On the Jetson Orin NX, the virtual-coordinate encoder and decoder require approximately $0.2\,\mathrm{ms}$ per control step, corresponding to about 1% of the $20\,\mathrm{ms}$ control period. Including the gain transformation of Eq. (19) and policy evaluation, the complete control thread requires approximately $1.10\,\mathrm{ms}$ for the proposed interface and remains compatible with the 50-Hz control rate. No control deadline was missed during any hardware rollout.

V. Conclusion

We introduced a body-aligned virtual joint interface that maps shared policy actions and observations to each robot's native joint space while preserving the existing transmission and low-level control stack. Across three humanoids with different native leg-axis layouts, a single unmodified recipe produced accurate, body-aligned, and stably trained velocity-tracking policies, whereas each baseline degraded on at least one platform. On Unitree G1 hardware, where simulation reports the smallest difference between interfaces, it reduced heading drift by nearly a factor of six at negligible onboard cost. A shared body-aligned convention can therefore reduce coordinate-induced embodiment dependence without per-robot tuning, and the benefit is understated by aggregate task metrics in simulation.

This study considered only legged velocity-tracking locomotion, with three seeds per configuration and hardware validation on a single platform. Future work will extend the interface to the arms, waist, and torso and investigate whole-body motion imitation and retargeting, enabling both training recipes and body-aligned reference motions to be reused across heterogeneous humanoids.

References

[1] J. Tan, T. Zhang, E. Coumans, A. Iscen, Y. Bai, D. Hafner, S. Bohez, and V. Vanhoucke, "Sim-to-real: Learning agile locomotion for quadruped robots," in *Robotics: Science and Systems XIV*, 2018.

[2] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, and M. Hutter, "Learning agile and dynamic motor skills for legged robots," *Science Robotics*, vol. 4, no. 26, p. eaau5872, 2019.

[3] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, "Learning quadrupedal locomotion over challenging terrain," *Science Robotics*, vol. 5, no. 47, p. eabc5986, 2020.

[4] T. Miki, J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, "Learning robust perceptive locomotion for quadrupedal robots in the wild," *Science Robotics*, vol. 7, no. 62, p. eabk2822, 2022.

[5] A. Kumar, Z. Fu, D. Pathak, and J. Malik, "RMA: Rapid motor adaptation for legged robots," in *Robotics: Science and Systems XVII*, 2021.

[6] N. Rudin, D. Hoeller, P. Reist, and M. Hutter, "Learning to walk in minutes using massively parallel deep reinforcement learning," in *Proceedings of the 5th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 164. PMLR, 2022, pp. 91–100. [Online]. Available: https://proceedings.mlr.press/v164/rudin22a.html

[7] Z. Xie, P. Clary, J. Dao, P. Morais, J. Hurst, and M. van de Panne, "Learning locomotion skills for Cassie: Iterative design and sim-to-real," in *Proceedings of the Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 100. PMLR, 2020, pp. 317–329. [Online]. Available: https://proceedings.mlr.press/v100/xie20a.html

[8] J. Siekmann, K. Green, J. Warila, A. Fern, and J. Hurst, "Blind bipedal stair traversal via sim-to-real reinforcement learning," in *Robotics: Science and Systems XVII*, 2021.

[9] I. Radosavovic, T. Xiao, B. Zhang, T. Darrell, J. Malik, and K. Sreenath, "Real-world humanoid locomotion with reinforcement learning," *Science Robotics*, vol. 9, no. 89, p. eadi9579, 2024.

[10] X. Gu, Y.-J. Wang, and J. Chen, "Humanoid-gym: Reinforcement learning for humanoid robot with zero-shot sim2real transfer," 2024, ICRA 2024 Workshop on Agile Robotics.

[11] J. Eßer, S. Kumar, H. Peters, V. Bargsten, J. de Gea Fernández, C. Mastalli, O. Stasse, and F. Kirchner, "Design, analysis and control of the series-parallel hybrid RH5 humanoid robot," in *2020 IEEE-RAS 20th International Conference on Humanoid Robots (Humanoids)*. IEEE, 2021, pp. 400–407.

[12] V. Lutz, L. de Matteis, V. Batto, and N. Mansard, "Control of humanoid robots with parallel mechanisms using differential actuation models," 2025.

[13] K. Fründ, A. L. Shu, F. C. Loeffl, and C. Ott, "A guideline for humanoid leg design with oblique axes for bipedal locomotion," in *2022 IEEE-RAS 21st International Conference on Humanoid Robots (Humanoids)*. IEEE, 2022, pp. 60–66.

[14] S. Sovukluk, J. Englsberger, and C. Ott, "Whole body control formulation for humanoid robots with closed/parallel kinematic chains: Kangaroo case study," in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2023, pp. 10 390–10 396.

[15] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017.

[16] X. B. Peng, P. Abbeel, S. Levine, and M. van de Panne, "DeepMimic: Example-guided deep reinforcement learning of physics-based character skills," *ACM Transactions on Graphics*, vol. 37, no. 4, pp. 143:1–143:14, 2018.

[17] H. Duan, J. Dao, K. Green, T. Apgar, A. Fern, and J. Hurst, "Learning task space actions for bipedal locomotion," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 1276–1282.

[18] D. Kim, G. Berseth, M. Schwartz, and J. Park, "Torque-based deep reinforcement learning for task-and-robot agnostic learning on bipedal robots using sim-to-real transfer," *IEEE Robotics and Automation Letters*, vol. 8, no. 10, pp. 6251–6258, 2023.

[19] T. Wang, R. Liao, J. Ba, and S. Fidler, "NerveNet: Learning structured policy with graph neural networks," in *International Conference on Learning Representations*, 2018. [Online]. Available: https://openreview.net/forum?id=S1sqHMZCb

[20] W. Huang, I. Mordatch, and D. Pathak, "One policy to control them all: Shared modular policies for agent-agnostic control," in *Proceedings of the 37th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 119. PMLR, 2020, pp. 4455–4464. [Online]. Available: https://proceedings.mlr.press/v119/huang20d.html

[21] A. Gupta, L. Fan, S. Ganguli, and L. Fei-Fei, "Metamorph: Learning universal controllers with transformers," in *International Conference on Learning Representations*, 2022.

[22] B. Trabucco, M. Phielipp, and G. Berseth, "AnyMorph: Learning transferable polices by inferring agent morphology," in *Proceedings of the 39th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 162. PMLR, 2022, pp. 21 677–21 691. [Online]. Available: https://proceedings.mlr.press/v162/trabucco22b.html

[23] Z. Xiong, J. Beck, and S. Whiteson, "Universal morphology control via contextual modulation," in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 202. PMLR, 2023, pp. 38 286–38 300. [Online]. Available: https://proceedings.mlr.press/v202/xiong23a.html

[24] N. Bohlinger, G. Czechmanowski, M. P. Krupka, P. Kicki, K. Walas, J. Peters, and D. Tateo, "One policy to run them all: An end-to-end learning approach to multi-embodiment locomotion," in *Proceedings of the 8th Conference on Robot Learning*, ser. Proceedings of Machine

Learning Research, vol. 270. PMLR, 2025, pp. 3356–3378. [Online]. Available: https://proceedings.mlr.press/v270/bohlinger25a.html

[25] Unitree Robotics, "Unitree RL Lab: Reinforcement learning environments for unitree robots," gitHub repository, accessed July 23, 2026. [Online]. Available: https://github.com/unitreerobotics/unitree_rl_lab

[26] E. Todorov, T. Erez, and Y. Tassa, "Mujoco: A physics engine for model-based control," in *2012 IEEE/RSJ international conference on intelligent robots and systems*. IEEE, 2012, pp. 5026–5033.